

Module 5 - Security, Dependencies, and Packaging

Grad Cafe database permissions

=====

Status

I ran the PostgreSQL-backed test suite and got 55 passes, no skipped tests, and 100% source coverage. This confirms that my configured test database accepted the operations exercised by those tests; it does not by itself confirm that the application connected as ``gradcafe_app`` or prove that role's effective grants.

I have not independently verified the live role privileges, ownership, or inherited memberships. I include the least-privileged SQL below as the role configuration I intend to use; I will treat it as verified only after checking the results of the read-only queries at the end of this file against my target database.

Runtime role

The intended runtime role is the non-superuser role ``gradcafe_app``. It should not own the table and should receive no CREATE, ALTER, or DROP privileges. Permissions for ``public.applicants``:

- * CONNECT on database ``gradcafe``: lets the user open a database connection.
- * USAGE on schema ``public``: lets the user resolve objects in that schema.
- * SELECT: reads applicant records for the analysis page, deduplication, and cleaning.
- * INSERT: adds records collected by the pull/load scripts.
- * UPDATE: saves normalized applicant values during cleaning.
- * TRUNCATE: supports the explicit database reset operation.
- * USAGE on ``applicants_p_id_seq``: allows inserts to generate serial ``p_id`` values.

The runtime role should not have schema CREATE, table ownership, ALTER, DROP, or superuser privileges. Verify the effective privileges rather than assuming the commands completed as intended.

Why packaging matters

``setup.py`` makes Module 5 installable as a Python project. Installing it with ``pip install -e .`` exposes the source modules consistently to local runs, tests, and CI instead of relying on the current working directory or ad-hoc ``PYTHONPATH`` settings. The setup metadata declares runtime dependencies and includes the Flask templates, static assets, and bundled applicant data needed by the installed application. Tooling dependencies are available with the ``dev`` extra (``pip install -e ".[dev]"``); ``requirements.txt`` remains the all-in-one environment installer. Tools such as uv can read dependencies from the project metadata when syncing an environment.

Dependency relationships

Flask serves the analysis page and connects its routes to the application logic. SQLAlchemy maps applicant records and supports ORM-based analysis queries. Psycopg runs parameterized application queries, while psycopg2 supports the legacy database setup script. `python-dotenv` loads local database settings without storing credentials in source code. BeautifulSoup parses Grad Cafe HTML, and urllib3 retrieves pages over HTTP. Pytest and pytest-cov test the project, Sphinx builds its documentation, and Pylint checks code quality. Finally, pydeps discovers module relationships and Graphviz renders them as a dependency diagram.

Security scan findings

Snyk Code initially reported low-severity path-traversal flows from the scraper CLI's input and output path arguments. The CLI now resolves those paths, checks that their canonical paths remain within the selected working directory, and rejects escapes, including symlinks that resolve outside it. A follow-up Snyk Code scan reported zero open issues. The CLI path checks are also covered by tests; rerun scans after code or dependency changes.

Separate setup and owner roles

`gradcafe\_setup` is a separate login role used only to initialize the table. Its temporary schema CREATE privilege is revoked after setup. The table and sequence are owned by `gradcafe\_owner`, a NOLOGIN, non-superuser role. Routine app execution must use `gradcafe\_app`, not either setup/owner role.

SQL excerpt

```
Run these statements as a PostgreSQL administrator, adjusting database/schema names if
needed. Set passwords interactively with `password`
CREATE ROLE gradcafe_app LOGIN NOSUPERUSER NOCREATEDB NOCREATEROLE
NOINHERIT;
CREATE ROLE gradcafe_setup LOGIN NOSUPERUSER NOCREATEDB NOCREATEROLE
NOINHERIT;
CREATE ROLE gradcafe_owner NOLOGIN NOSUPERUSER NOCREATEDB
NOCREATEROLE NOINHERIT;
GRANT CONNECT ON DATABASE gradcafe TO gradcafe_app, gradcafe_setup;
password gradcafe_app
password gradcafe_setup
connect gradcafe
REVOKE CREATE ON SCHEMA public FROM PUBLIC;
GRANT USAGE ON SCHEMA public TO gradcafe_app;
GRANT USAGE, CREATE ON SCHEMA public TO gradcafe_setup;
```

After creating `public.applicants` using the setup account, the reported setup applies the runtime grants, transfers ownership, and disables setup access:

```
GRANT SELECT, INSERT, UPDATE, TRUNCATE
  ON TABLE public.applicants TO gradcafe_app;
GRANT USAGE ON SEQUENCE public.applicants_p_id_seq TO gradcafe_app;
GRANT CREATE ON SCHEMA public TO gradcafe_owner;
ALTER TABLE public.applicants OWNER TO gradcafe_owner;
ALTER SEQUENCE public.applicants_p_id_seq OWNER TO gradcafe_owner;
REVOKE CREATE ON SCHEMA public FROM gradcafe_setup;
REVOKE CREATE ON SCHEMA public FROM gradcafe_owner;
REVOKE CONNECT ON DATABASE gradcafe FROM gradcafe_setup;
ALTER ROLE gradcafe_setup NOLOGIN;
```

Verify actual database state

Run these read-only checks while connected to the target `gradcafe` database as a PostgreSQL administrator. They report role flags, effective permissions, and the table/sequence owners; the final query lists any roles inherited by the runtime account.

```
SELECT rolname, rolsuper, rolcreatedb, rolcreaterole, rolcanlogin
FROM pg_roles
WHERE rolname IN ('gradcafe_app', 'gradcafe_setup', 'gradcafe_owner');
```

```
SELECT
  has_database_privilege('gradcafe_app', 'gradcafe', 'CONNECT') AS can_connect,
  has_schema_privilege('gradcafe_app', 'public', 'USAGE') AS schema_usage,
  has_schema_privilege('gradcafe_app', 'public', 'CREATE') AS schema_create;
```

```
SELECT
  pg_get_userbyid(c.relowner) AS table_owner,
  has_table_privilege('gradcafe_app', c.oid, 'SELECT') AS can_select,
  has_table_privilege('gradcafe_app', c.oid, 'INSERT') AS can_insert,
  has_table_privilege('gradcafe_app', c.oid, 'UPDATE') AS can_update,
  has_table_privilege('gradcafe_app', c.oid, 'TRUNCATE') AS can_truncate,
  has_table_privilege('gradcafe_app', c.oid, 'DELETE') AS can_delete
FROM pg_class AS c
JOIN pg_namespace AS n ON n.oid = c.relnamespace
WHERE n.nspname = 'public' AND c.relname = 'applicants';
```

```
SELECT
  pg_get_userbyid(s.relowner) AS sequence_owner,
  has_sequence_privilege('gradcafe_app', s.oid, 'USAGE') AS sequence_usage,
  has_sequence_privilege('gradcafe_app', s.oid, 'UPDATE') AS sequence_update
FROM pg_class AS s
```

```
WHERE s.oid = pg_get_serial_sequence('public.applicants', 'p_id')::regclass;
```

```
SELECT parent.rolname AS inherited_role  
FROM pg_auth_members AS m  
JOIN pg_roles AS parent ON parent.oid = m.roleid  
JOIN pg_roles AS member ON member.oid = m.member  
WHERE member.rolname = 'gradcafe_app';
```

Also inspect ``du``, ``dp public.applicants``, and
``dp public.applicants\_p\_id\_seq``. The expected result is a non-superuser ``gradcafe\_app`` with
CONNECT and schema USAGE, SELECT/INSERT/UPDATE/TRUNCATE and sequence
USAGE, no schema CREATE or DELETE, no ownership, and no inherited memberships that
grant extra privileges. ``gradcafe\_setup`` should be NOLOGIN; ``gradcafe\_owner`` should be
NOLOGIN and own the table and sequence. Check schema ACLs for unintended PUBLIC
CREATE grants as well.